# Identificación y clasificación de especies de mosquitos en imágenes: revisión bibliográfica y selección preliminar de algoritmos

Proyecto 2

CC3084 Data Science

Universidad del Valle de Guatemala

Octubre de 2026

## Identificación y clasificación de especies de mosquitos en imágenes

### Contexto del problema

Este proyecto parte del reto MosquitoAlert Challenge 2023, en el que hay que encontrar un mosquito dentro de una fotografía tomada por una persona con su celular y decir a cuál de seis categorías pertenece: *Aedes aegypti*, *Aedes albopictus*, *Anopheles*, *Culex*, *Culiseta* o el complejo *Aedes japonicus/koreicus* (Mosquito Alert Team y AIcrowd, 2023). El conjunto de entrenamiento trae 8,025 imágenes con una caja delimitadora por imagen, anotadas por entomólogos. La métrica del reto es el F1 macro, con una condición adicional: si la caja que predecimos tiene una intersección sobre unión (IoU) menor a 0.75 respecto a la caja real, esa imagen no cuenta para la clasificación. Es decir, hay que localizar bien y clasificar bien.

Dos rasgos del conjunto pesan mucho en todas las decisiones que siguen. El primero es el desbalance. *A. albopictus* y *Culex* suman cerca del 89 % de las imágenes, mientras que *A. aegypti* tiene apenas 38 (unas 94 veces menos que *A. albopictus*). Como el F1 macro le da el mismo peso a cada clase, las seis valen igual aunque una tenga 38 ejemplos y otra más de 3,500 (Tabla 1). El segundo rasgo es la naturaleza de las fotos. Vienen de ciencia ciudadana, así que el mosquito ocupa una parte pequeña de la imagen y cambian la pose, la luz y el fondo. Pataki et al. (2021) apuntan que la calidad de las fotos de los voluntarios es una limitación del sistema y que revisar cada imagen a mano no es sostenible a largo plazo, que es justo lo que motiva automatizar el proceso.

**Tabla 1**

*Distribución de clases en el conjunto de entrenamiento*

| Clase | Imágenes | Proporción | Nivel taxonómico |
|---|---|---|---|
| *Aedes albopictus* | 3,567 | 44.4 % | Especie |
| *Culex* | 3,544 | 44.2 % | Género |
| *Culiseta* | 492 | 6.1 % | Género |
| *Aedes japonicus/koreicus* | 321 | 4.0 % | Complejo de especies |
| *Anopheles* | 63 | 0.8 % | Género |
| *Aedes aegypti* | 38 | 0.5 % | Especie |
| Total | 8,025 | 100 % | |

*Nota.* Elaboración propia a partir de la descripción del reto (Mosquito Alert Team y AIcrowd, 2023).

## Revisión bibliográfica

### Clasificación de mosquitos con redes neuronales convolucionales

Casi todos los trabajos que encontramos usan redes neuronales convolucionales (CNN), casi siempre con transferencia de aprendizaje. Motta et al. (2019) entrenaron LeNet, AlexNet y GoogLeNet con 4,056 imágenes de *A. aegypti*, *A. albopictus* y *Culex quinquefasciatus* capturadas en campo; la exactitud de validación fue de 57.5 %, 74.7 % y 83.9 %, respectivamente, así que la arquitectura importó bastante. En un segundo trabajo, Motta et al. (2020) probaron Xception y DenseNet con una búsqueda de hiperparámetros (aleatoria y en rejilla) sobre 7,561 imágenes de mosquitos y 1,187 de otros insectos, y llegaron a una exactitud balanceada de 93.5 % para separar mosquitos de otros insectos y de 97.3 % para distinguir *Aedes* de *Culex*.

Park et al. (2020) clasificaron mosquitos vectores con CNN profundas y además visualizaron qué regiones de la imagen usaba el modelo; concluyeron que se apoya en rasgos morfológicos parecidos a los que usan los expertos, lo cual da confianza en que la red no está aprendiendo atajos. Ong et al. (2021) separaron *A. aegypti* de *A. albopictus* con una MobileNet preentrenada (4,120 imágenes) y superaron el 98 %, sin diferencia significativa frente a entomólogos, aunque con fotos tomadas en un dispositivo estandarizado, algo muy distinto de lo que tenemos nosotros. La Tabla 2 resume estos estudios.

**Tabla 2**

*Estudios de clasificación de mosquitos con aprendizaje profundo*

| Estudio | Datos | Enfoque | Resultado reportado |
|---|---|---|---|
| Motta et al. (2019) | 4,056 imágenes; 3 especies | LeNet, AlexNet, GoogLeNet | 57.5 %, 74.7 % y 83.9 % de exactitud |
| Motta et al. (2020) | 7,561 mosquitos y 1,187 otros insectos | Xception y DenseNet con búsqueda de hiperparámetros | Exactitud balanceada de 93.5 % y 97.3 % |
| Park et al. (2020) | Imágenes de mosquitos vectores | CNN profundas y análisis de regiones discriminativas | Rasgos aprendidos similares a los de los expertos |
| Kittichai et al. (2021) | 10,564 imágenes; 15 clases | YOLO v2 y v3, una y dos etapas | Dos etapas superó a una etapa |
| Ong et al. (2021) | 4,120 imágenes; 2 especies | MobileNet con transferencia | Más de 98 %, equivalente a expertos |

*Nota.* Las cifras son las que reportan los autores en cada estudio; no son comparables entre sí porque los datos y las condiciones de captura son distintos.

**Detección y localización del mosquito**

Como el reto exige un IoU de al menos 0.75, la detección forma parte del problema. Kittichai et al. (2021) trabajaron con un caso parecido, objetos pequeños dentro de imágenes grandes, usando la familia YOLO. Compararon una sola etapa (una red que localiza y clasifica a la vez) contra dos etapas encadenadas: una primera red detecta el mosquito, se recorta esa región y una segunda red identifica género, especie y sexo. Con cada una de las variantes de YOLO que probaron, la versión de dos etapas rindió mejor, y la mejor de todas fue YOLO v3 en dos etapas. También notaron que el desempeño por clase seguía de cerca el número de imágenes de entrenamiento, con las detecciones más bajas en las clases con pocas muestras, y sugieren reunir más de unas 300 imágenes sin aumentar por clase. Eso nos toca de cerca, porque dos de nuestras clases tienen 38 y 63.

Pataki et al. (2021) entrenaron un modelo con la base curada de Mosquito Alert, la misma fuente de nuestros datos, para reconocer al mosquito tigre (*A. albopictus*) en fotos ciudadanas, y exploraron usarlo para dar retroalimentación que mejore las fotos que se envían. Para escoger la versión del detector, Terven et al. (2023) hacen un recorrido por la evolución de YOLO, desde su primera versión hasta YOLOv8 y YOLO-NAS, con las métricas estándar y los cambios de arquitectura y de entrenamiento de cada generación.

**Transferencia de aprendizaje y modelos preentrenados**

En mosquitos, lo habitual es partir de redes entrenadas en ImageNet (Motta et al., 2020; Ong et al., 2021). Una alternativa más reciente son los modelos fundacionales de visión. Oquab et al. (2024) muestran que DINOv2, entrenado sin supervisión con datos curados, produce características de uso general que sirven sin ajuste fino, y que sus versiones más pequeñas superan a OpenCLIP en la mayoría de los *benchmarks* que evaluaron. Para nosotros eso abre la posibilidad de extraer características una sola vez y entrenar encima un clasificador sencillo, lo cual es barato y puede ayudar con las clases de muy pocos ejemplos.

**Desbalance de clases**

Buda et al. (2018) compararon de forma sistemática sobremuestreo, submuestreo, entrenamiento en dos fases y ajuste de umbrales en CNN. Encontraron que el desbalance perjudica, que el sobremuestreo fue el mejor método en casi todos los escenarios, que conviene llevarlo hasta eliminar el desbalance y que, a diferencia de los modelos clásicos, no provocó sobreajuste en las CNN.

**Clasificación jerárquica**

Silla y Freitas (2011) revisan la clasificación jerárquica en distintos dominios, proponen un marco unificador para ordenar los enfoques existentes y comparan sus ventajas y desventajas. No es una revisión de visión por computadora, pero nos sirve de punto de partida. Por nuestra cuenta, lo que vemos como riesgo de separar por niveles es que un error en el nivel de arriba ya no se puede corregir abajo. Esto nos importa porque nuestras clases sí tienen una jerarquía natural, la taxonómica; tres de las seis pertenecen al género *Aedes*.

### Lo que deja la literatura

Al juntar todo, vemos tres cosas. La mayoría de los estudios usa imágenes de laboratorio o capturadas con dispositivos controlados, y pocos trabajan con fotos ciudadanas, que es lo que tenemos. Los conjuntos reportados suelen estar balanceados o tener decenas o cientos de ejemplos por clase; nuestro desbalance es más severo. Y la evidencia sobre YOLO sugiere que separar detección y clasificación ayuda con objetos pequeños (Kittichai et al., 2021). Además, en las fuentes que revisamos no encontramos una comparación directa entre clasificación plana y jerárquica en mosquitos, así que esa comparación podría ser una contribución propia del proyecto.

## Selección preliminar de algoritmos

Con base en lo anterior, planteamos un sistema de dos etapas: primero un detector que encuentra al mosquito, luego un clasificador que trabaja sobre el recorte. Para la segunda etapa vamos a comparar tres candidatos. Todo esto es preliminar; lo confirmaremos o ajustaremos con el análisis exploratorio y los primeros experimentos.

### Detector

Para la primera etapa elegimos YOLO, en sus variantes pequeñas (*n* y *s*), entrenado con una sola clase: “mosquito”. Tiene sentido por varias razones. Kittichai et al. (2021) vieron que separar localización y clasificación mejora el resultado con mosquitos pequeños. Al juntar las seis clases en una, aprovechamos las 8,025 cajas y el desbalance deja de afectar al detector, que es lo que necesitamos para pasar el umbral de IoU del reto. Como cada imagen tiene una sola caja, basta con quedarnos con la detección de mayor confianza. Y las variantes pequeñas de YOLO (Terven et al., 2023) caben en la tarjeta gráfica que tenemos, una GTX 1650 de 4 GB.

### Clasificador

Para la segunda etapa vamos a comparar tres opciones, de menor a mayor costo. La primera es EfficientNet-B0, que será la línea base: es ligera, se usa mucho en clasificación fina con transferencia y se parece a las arquitecturas que ya funcionaron en mosquitos (Motta et al., 2020; Ong et al., 2021). La segunda, y nuestro modelo principal, es ConvNeXt-Tiny, una CNN moderna que adopta decisiones de diseño de los transformadores sin dejar de ser convolucional; esperamos que discrimine mejor que la línea base, aunque en 4 GB habrá que usar batch pequeño con acumulación de gradiente o pasar parte de los experimentos a Colab. La tercera es DINOv2 (ViT-S/14) congelado con un clasificador lineal encima, regresión logística o SVM (Oquab et al., 2024). Es la más barata y la que más nos interesa para *A. aegypti* y *Anopheles*, donde no hay datos para ajustar una red completa.

Dejamos fuera, como línea principal, los modelos clásicos con características manuales (SVM o bosques aleatorios), que la literatura reciente ha superado con CNN en imágenes de campo, y los transformadores de visión con ajuste fino completo, porque piden más memoria de la que tenemos. Ambos pueden servir como punto de comparación.

**Manejo del desbalance**

Siguiendo a Buda et al. (2018), combinaremos tres cosas: muestreo balanceado por clase o sobremuestreo hasta eliminar el desbalance, con aumento de datos fuerte en las clases minoritarias; pérdida ponderada por clase y, como variante, *focal loss*; y un ajuste de umbrales al final para optimizar el F1 macro. Haremos la partición en entrenamiento, validación y prueba de forma estratificada y *antes* de cualquier sobremuestreo, para no colar copias de una misma imagen en particiones distintas. Evaluaremos con F1 macro, recall por clase y matriz de confusión; la exactitud global no dice mucho con un desbalance así.

**Nuestra propuesta jerárquica**

La idea que tenemos es un primer modelo con tres salidas (*albopictus*, *Culex* y "otro") y un segundo modelo, que solo corre cuando el primero responde "otro", para separar *Culiseta*, *japonicus/koreicus*, *Anopheles* y *aegypti*. Es un esquema de clasificación jerárquica en cascada (Silla y Freitas, 2011). Lo mantenemos como candidato, pero con dos reservas que salen de los datos. La primera es la propagación de errores: *A. aegypti* se parece mucho a *A. albopictus*, que tiene unas 94 veces más ejemplos, así que es probable que el primer modelo mande muchos *aegypti* a *albopictus* y el segundo nunca los vea; con F1 macro eso cuesta caro. La segunda es que el segundo nivel sigue desbalanceado: el grupo "otro" tiene 914 imágenes (11.4 %) repartidas en 492, 321, 63 y 38.

Por eso vamos a comparar tres variantes con el mismo backbone: una plana de seis clases; una jerárquica por taxonomía, con un primer nivel {*Aedes*, *Culex*, *Culiseta*, *Anopheles*} y un segundo nivel que solo separa las especies de *Aedes* (así las clases más parecidas quedan juntas); y la cascada que propusimos, con {*albopictus*, *Culex*, otro}. Si resulta viable, la implementaremos con un backbone compartido y dos cabezas entrenadas a la vez, lo que evita encadenar errores. Con esto cubrimos la brecha que mencionamos antes.

**Recursos disponibles**

Contamos con una laptop con GPU GTX 1650 (4 GB), 16 GB de RAM y un procesador i5, y con acceso a Google Colab. En local son viables el detector YOLO pequeño, EfficientNet-B0 y la extracción de características con DINOv2; ConvNeXt-Tiny queda justo y lo correremos con batch reducido o en Colab. Como el conjunto completo pesa cerca de 9 GB, vamos a precalcular los recortes una sola vez, a baja resolución, y reutilizarlos en todos los experimentos. Para que el clasificador no se acostumbre a recortes perfectos, lo entrenaremos con la caja anotada más un margen y pequeños desplazamientos aleatorios, y la evaluación final la haremos con los recortes que produzca el detector y no con las cajas manuales.

## Limitaciones

Con 38 imágenes de *A. aegypti* y 63 de *Anopheles*, cualquier partición deja muy pocos ejemplos de prueba por clase, así que las métricas de esas dos clases tendrán mucha varianza y lo reportaremos con intervalos o resultados por pliegue. Además, casi todos los estudios revisados usan imágenes más controladas que las fotos ciudadanas, por lo que sus cifras no se pueden comparar directamente con las que obtengamos.

## Referencias


Buda, M., Maki, A., & Mazurowski, M. A. (2018). A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks, 106,* 249–259. https://doi.org/10.1016/j.neunet.2018.07.011

Kittichai, V., Pengsakul, T., Chumchuen, K., Samung, Y., Sriwichai, P., Phatthamolrat, N., Tongloy, T., Jaksukam, K., Chuwongin, S., & Boonsang, S. (2021). Deep learning approaches for challenging species and gender identification of mosquito vectors. *Scientific Reports, 11,* Article 4838. https://doi.org/10.1038/s41598-021-84219-4

Mosquito Alert Team, & AIcrowd. (2023). *MosquitoAlert Challenge 2023* [Descripción del reto y conjunto de datos]. AIcrowd. https://www.aicrowd.com/challenges/mosquitoalert-challenge-2023

Motta, D., Santos, A. Á. B., Winkler, I., Machado, B. A. S., Pereira, D. A. D. I., Cavalcanti, A. M., Fonseca, E. O. L., Kirchner, F., & Badaró, R. (2019). Application of convolutional neural networks for classification of adult mosquitoes in the field. *PLOS ONE, 14*(1), Article e0210829. https://doi.org/10.1371/journal.pone.0210829

Motta, D., Santos, A. Á. B., Machado, B. A. S., Ribeiro-Filho, O. G. V., Camargo, L. O. A., Valdenegro-Toro, M. A., Kirchner, F., & Badaró, R. (2020). Optimization of convolutional neural network hyperparameters for automatic classification of adult mosquitoes. *PLOS ONE, 15*(7), Article e0234959. https://doi.org/10.1371/journal.pone.0234959

Ong, S.-Q., Ahmad, H., Nair, G., Isawasan, P., & Majid, A. H. A. (2021). Implementation of a deep learning model for automated classification of *Aedes aegypti* (Linnaeus) and *Aedes albopictus* (Skuse) in real time. *Scientific Reports, 11,* Article 9908. https://doi.org/10.1038/s41598-021-89365-3

Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M., Sharma, V., … Bojanowski, P. (2024). DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*. https://doi.org/10.48550/arXiv.2304.07193

Park, J., Kim, D. I., Choi, B., Kang, W., & Kwon, H. W. (2020). Classification and morphological analysis of vector mosquitoes using deep convolutional neural networks. *Scientific Reports, 10,* Article 1012. https://doi.org/10.1038/s41598-020-57875-1

Pataki, B. A., Garriga, J., Eritja, R., Palmer, J. R. B., Bartumeus, F., & Csabai, I. (2021). Deep learning identification for citizen science surveillance of tiger mosquitoes. *Scientific Reports, 11,* Article 4718. https://doi.org/10.1038/s41598-021-83657-4

Silla, C. N., Jr., & Freitas, A. A. (2011). A survey of hierarchical classification across different application domains. *Data Mining and Knowledge Discovery*, *22*(1–2), 31–72. https://doi.org/10.1007/s10618-010-0175-9

Terven, J., Córdova-Esparza, D.-M., & Romero-González, J.-A. (2023). A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction*, *5*(4), 1680–1716. https://doi.org/10.3390/make5040083